How SQL, Spark SQL, and PySpark Compare Strings

1. Do they convert strings to hexadecimal before comparing?

No. SQL, Spark SQL, and PySpark do not normally convert strings into hexadecimal before comparing them. They compare string values according to the engine's comparison rules. Behavior can depend on the database, data type, collation, and operation.

Hexadecimal is useful when *we* want to inspect the underlying characters and identify invisible differences, such as a normal space versus a non-breaking space.

2. Normal space versus non-breaking space

Consider two values that look almost identical:

• String A: `Hello World` — contains a normal space (`U+0020`).

• String B: `Hello World` — contains a non-breaking space (`U+00A0`).

Their UTF-8 encodings differ:

Character	Unicode code point	UTF-8 bytes
Normal space	`U+0020`	`20`
Non-breaking space	`U+00A0`	`C2 A0`

When the engine compares strings, these different characters can make the values unequal. Exact behavior can vary by engine and collation.

3. How string comparison works

Conceptually:

1. The engine receives the two string values.

2. It applies the relevant string comparison rules. Depending on the engine, these can involve encoding, data type, collation, case sensitivity, and padding semantics.

3. It returns a comparison result, such as equal or not equal.

Hexadecimal is a representation used to inspect character data. It is not a mandatory intermediate format for every SQL comparison. At a lower level, a system may operate on encoded bytes, character code units, or collation weights, depending on its implementation.

4. How `=`, `<>`, `LIKE`, and substring checks behave

For example:

```
SELECT 'Hello World' = 'Hello World'; -- true
SELECT 'Hello World' = 'Hello  World'; -- false: two spaces
```

The second string has two normal spaces instead of one, so the values differ under ordinary binary-style comparison.

Operation	What it checks
`=`	Whether two strings compare as equal
`<>`	Whether two strings compare as unequal
`LIKE`	Whether a string matches a pattern
`contains()` / substring functions	Whether a substring occurs inside a string
`JOIN ... ON a.name = b.name`	Whether join keys compare as equal

In Spark SQL:

```
SELECT *
FROM customers
WHERE name = 'John Smith';
```

If the stored value has a non-breaking space between `John` and `Smith`, this filter can fail to match the normal-space version.

But:

```
SELECT *
FROM customers
WHERE name LIKE '%Smith%';
```

can still match, because the pattern looks for `Smith` anywhere in the string. The hidden space does not itself prevent that substring from matching.

Important: trailing spaces, case sensitivity, and collation behavior can differ between database engines. Do not assume every SQL engine compares every string identically.

5. Inspecting hexadecimal in Spark SQL / Databricks SQL

To explicitly convert a string to UTF-8 bytes and display those bytes as hexadecimal:

```
SELECT
    name,
    hex(encode(name, 'UTF-8')) AS hex_value
FROM customers;
```

Example UTF-8 representations:

```
Hello World  -> 48656C6C6F20576F726C64
Hello World  -> 48656C6C6F C2A0 576F726C64
```

The spaces in the second hexadecimal value are only for readability. `20` represents the normal space; `C2A0` represents the non-breaking space in UTF-8.

This query explicitly requests hexadecimal output for debugging. A normal `WHERE name = ...` comparison does not need this conversion.

6. What happens in PySpark?

PySpark DataFrame operations use Spark's execution engine. They do not normally convert strings to hexadecimal for comparisons.

```
from pyspark.sql import functions as F

# Equality
df.filter(F.col("name") == "John Smith")

# SQL LIKE pattern
df.filter(F.col("name").like("%Smith%"))

# Find a substring
df.filter(F.instr(F.col("name"), "Smith") > 0)
```

To inspect UTF-8 bytes as hexadecimal:

```
df.select(
    "name",
    F.hex(F.encode(F.col("name"), "UTF-8")).alias("hex_value")
).show(truncate=False)
```

Here, hexadecimal is explicitly requested for troubleshooting; the filter itself does not require it.

7. Practical checklist for data engineering

When a SQL or Spark join/filter unexpectedly misses records:

1. Check whether the values actually differ, including leading/trailing spaces and invisible Unicode characters.
2. Inspect the UTF-8 hex representation when needed.
3. Check the engine's collation, case-sensitivity, and trailing-space behavior.
4. Normalize known unwanted characters before joining or filtering, using an explicit and tested rule.
5. Avoid blindly removing all whitespace: spaces can be meaningful in names, addresses, identifiers, and free-text fields.
6. Test cleanup on representative source data and validate row counts and join-match rates afterward.

Interview-ready answer

SQL, Spark SQL, and PySpark do not ordinarily convert strings to hexadecimal before comparing them. The engine evaluates string values according to its comparison semantics, which may include encoding and collation rules. Hexadecimal is a debugging technique that lets us inspect the encoded bytes and find hidden differences, such as `U+0020` versus `U+00A0`. In Spark, `hex(encode(col, 'UTF-8'))` is one way to inspect a string's UTF-8 bytes.

Further reading

• [Spark SQL string functions](https://spark.apache.org/docs/latest/api/sql/string-functions/)

• [Databricks SQL functions](https://docs.databricks.com/en/sql/language-manual/sql-ref-functions.html)